Appliance Energy Prediction Using Deep Learning

Subject:

Multivariate Time-Series Forecasting

Objective:

Predict appliance energy consumption using environmental and temporal features using Machine Learning and Deep Learning models.

Name:

Jithara Siriwardana

Submission Date:

[06/08/2026]

Contents

1 Introduction

1.1 Background

Energy consumption forecasting is an important task in smart building management. Accurate prediction of appliance energy usage helps optimize energy efficiency, reduce costs, and support sustainable energy practices.

The Appliance Energy Prediction dataset contains environmental, weather, and indoor sensor measurements collected at 10-minute intervals. The objective of this project is to predict appliance energy consumption using deep learning techniques.

1.2 Objectives

The main objectives of this project are:

- Perform exploratory data analysis on the dataset.
- Clean and preprocess the data.
- Engineer meaningful time-series features.
- Develop a deep learning LSTM model.
- Optimize model performance using hyperparameter tuning.
- Compare model performance using standard evaluation metrics.

2 Dataset Description

2.1 Dataset Overview

The Appliance Energy Prediction dataset contains approximately 20,000 observations collected at 10-minute intervals.

Target Variable:

- `Appliances` – Energy consumption in Wh

Input Features:

- Indoor temperatures (`T1`–`T6`)
- Indoor humidity measurements (`RH_1`–`RH_6`)
- Outdoor temperature
- Outdoor humidity
- Wind speed
- Visibility
- Atmospheric pressure
- Lighting energy consumption
- Date and time information

2.2 Dataset Inspection

The dataset was loaded into Pandas and inspected using:

- `head()`
- `info()`
- `describe()`

These functions provided information regarding data types, missing values, and statistical distributions.

3 Exploratory Data Analysis (EDA)

3.1 Purpose of EDA

EDA was performed to understand:

- Data distributions
- Trends over time
- Correlations among variables
- Potential anomalies and outliers

3.2 Energy Consumption Trend

A time-series plot of appliance energy consumption was created.

Observations:

- Energy usage fluctuates significantly over time.
- Several peaks indicate periods of high energy demand.
- Seasonal and cyclical patterns are visible.

3.3 Correlation Analysis

A correlation heatmap was generated.

Observations:

- Indoor temperatures show positive correlations with energy usage.
- Humidity variables exhibit moderate relationships.
- Lighting consumption is correlated with appliance energy consumption.

3.4 Distribution Analysis

Histograms and boxplots were generated.

Observations:

- Appliance energy consumption is right-skewed.
- Extreme values are present, indicating potential outliers.

4 Data Preprocessing

This section explains the complete preprocessing workflow used to transform the raw Appliance Energy Prediction dataset into a clean, structured, and model-ready time-series dataset.

4.1 Data Loading and Initial Inspection

The dataset was loaded using Pandas and inspected to understand its structure, shape, and feature types.

Steps Performed:

- Loaded the dataset using `pd.read_csv()`.
- Checked the dataset shape to identify the number of rows and columns.
- Inspected column types and dataset structure.

Purpose:

Initial inspection helps understand the raw data format before applying preprocessing transformations.

4.2 Date-Time Conversion

The `date` column was converted into datetime format to enable time-series feature extraction.

Why This Is Important:

- Machine learning models cannot directly interpret raw timestamp strings.
- Date-time conversion enables extraction of useful time-based features such as hour, day, month, and weekday.

4.3 Missing Value Handling

Missing values were analyzed using `df.isnull().sum()`.

Findings:

Check	Result
Missing values	0

Strategy Used:

- Interpolation method: `df.interpolate()`.

Why Interpolation Was Used:

- Maintains temporal continuity in the time-series data.
- Preserves trend information instead of dropping rows unnecessarily.
- Is suitable for sensor-based datasets where neighboring observations are often related.

4.4 Time-Based Feature Engineering During Preprocessing

From the datetime column, multiple time-based features were extracted.

Features Created:

Feature	Description
`hour`	Hour of day
`day`	Day of month
`month`	Month of year
`weekday`	Day of week
`is_weekend`	Weekend indicator

Why These Are Important:

Energy consumption depends strongly on time of day, weekend versus weekday behavior, and seasonal variation. These features help the model learn recurring temporal patterns such as morning and evening energy peaks.

4.5 Rolling Window Features

Rolling statistics were created to capture short-term trends in appliance energy consumption.

Features Created:

- Rolling mean over 1 hour using 6 time steps.
- Rolling mean over 3 hours using 18 time steps.

Purpose:

- Smooth short-term fluctuations.
- Capture local energy consumption trends.
- Reduce noise in the time-series data.

4.6 Lag Features

Lag features were created to capture past dependencies in appliance energy consumption.

Features Created:

Feature	Meaning
`lag_1`	Previous 10 minutes
`lag_3`	Previous 30 minutes
`lag_6`	Previous 60 minutes

Why Lag Features Matter:

- Energy consumption depends strongly on previous usage behavior.
- Lag values help the model learn temporal dependencies.
- They provide useful historical context for forecasting future appliance energy consumption.

4.7 Interaction Features

An interaction feature was created by combining temperature and humidity:

$$\texttt{Temperature Humidity Interaction} = T1 \times RH\_1$$

Purpose:

- Captures combined environmental effects.
- Represents situations where high humidity and high temperature may increase appliance or cooling-related energy usage.

4.8 Handling NaN Values After Feature Engineering

After creating lag features and rolling window features, new missing values were introduced at the beginning of the time series.

Solution:

- Removed missing values using `df.dropna()`.

Reason:

- Ensures a clean dataset for model training.
- Prevents errors when reshaping data for LSTM input.
- Removes rows where lag or rolling feature values are not available.

4.9 Outlier Detection and Treatment

Outliers were handled using the Interquartile Range (IQR) method.

Steps:

- Compute Q_1, the 25th percentile.
- Compute Q_3, the 75th percentile.
- Calculate the interquartile range: $\text{IQR} = Q_3 - Q_1$.

Outlier Rule:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR},$$
$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}.$$

Values below the lower bound or above the upper bound were treated as outliers.

Treatment Method:

- Outliers were capped rather than removed.

Why Capping Was Used:

- Preserves dataset size.
- Prevents extreme values from distorting the distribution.
- Maintains time-series continuity, which is important for sequential modeling.

4.10 Feature Scaling

Although scaling may be applied during the modeling stage, it is an important preprocessing step for deep learning models.

Techniques Used:

- Standardization or Min-Max Scaling, depending on the model requirement.

Purpose:

- Normalizes feature ranges.
- Improves deep learning convergence.
- Prevents large-scale features from dominating smaller-scale features.

For Min-Max Scaling, the transformation is:

$$\text{Scaled Value} = \frac{x - \min}{\max - \min}.$$

4.11 Final Dataset Preparation

After preprocessing, all transformations were applied to produce a clean and model-ready dataset.

Final Steps:

- Applied all preprocessing transformations.
- Removed missing values introduced by rolling and lag features.
- Handled outliers using capping.
- Added time-based, rolling, lag, and interaction features.

Final Output:

- Clean processed dataset saved as `data/processed/processed_energy_data.csv`.

5 Feature Selection

5.1 Importance Analysis

Feature importance analysis was used to identify the most influential predictors.

Reason:

Feature importance analysis helps identify variables that contribute most strongly to prediction performance.

5.2 Selected Features

Top 15 most important features were retained.

Benefits:

- Reduced dimensionality
- Faster training
- Improved generalization

6 Deep Learning Model

6.1 Why LSTM?

Long Short-Term Memory (LSTM) networks are designed for sequential data and can capture long-term dependencies in time-series datasets.

Advantages:

- Learns temporal patterns
- Handles long sequences
- Effective for forecasting problems

6.2 LSTM Architecture

Layer Structure:

- LSTM (128 units)
- Dropout (0.3)
- LSTM (64 units)
- Dropout (0.3)
- Dense (32)
- Dense (1)

Activation Functions:

- ReLU in dense layer

Optimizer:

- Adam

Loss Function:

- Mean Squared Error (MSE)

7 Hyperparameter Tuning

7.1 Introduction

Hyperparameter tuning is a crucial step in deep learning model optimization. Unlike model parameters, which are weights learned during training, hyperparameters are predefined settings that control the learning process and model architecture. Proper tuning improves prediction accuracy, reduces overfitting, and supports better generalization on unseen data.

In this project, hyperparameter tuning was applied to the LSTM-based multivariate time-series forecasting model to optimize appliance energy consumption prediction performance.

7.2 Key Hyperparameters Considered

The following hyperparameters were identified as the most influential for model performance.

Learning Rate:

The learning rate controls how quickly the model updates its weights during training. A high learning rate may cause unstable training, while a very low learning rate may result in slow convergence.

- Tested values: 0.001, 0.0005, and 0.0001.
- Final selected value: 0.001.

Batch Size:

Batch size defines the number of samples processed before updating model weights. It affects training stability, convergence behavior, and memory usage.

- Tested values: 16, 32, and 64.
- Final selected value: 32.

Number of LSTM Layers:

The number of LSTM layers controls model depth and its ability to learn temporal patterns.

- 1 LSTM layer: showed underfitting.
- 2 LSTM layers: provided the best balance between learning capacity and generalization.
- 3 LSTM layers: increased the risk of overfitting.

Number of Units:

The number of units, or neurons, determines the learning capacity of each LSTM layer.

- Tested configurations: 64, 128, and 256 units.
- 128 units performed best in the first LSTM layer.
- 256 units showed signs of overfitting.

Dropout Rate:

Dropout was used to reduce overfitting by randomly disabling a fraction of neurons during training.

- Tested values: 0.1, 0.2, and 0.3.
- Final selected value: 0.2.

Epochs:

Epochs represent the number of complete passes through the training dataset. Early stopping was used to avoid unnecessary training once validation loss stopped improving.

- Maximum epochs tested: 30–50.
- Early stopping was used to select the best training point automatically.

7.3 Optimization Strategy

To efficiently find the best hyperparameter combination, several optimization strategies were considered.

Manual Tuning:

Initial experiments were performed by changing one parameter at a time. This helped identify a strong baseline model configuration before testing more refined settings.

Early Stopping:

Early stopping was applied to stop training when validation loss stopped improving.

- Monitor: validation loss.
- Patience: 5 epochs.
- `restore_best_weights`: True.

This prevented overfitting by automatically restoring the model weights from the epoch with the best validation performance.

Grid Search:

Different hyperparameter combinations were evaluated systematically in selected experiments. A full grid search was not applied because of computational cost, but selected combinations were tested manually to compare model behavior.

Keras Tuner Extension:

As an advanced improvement, Keras Tuner can be used to automatically search for better hyperparameters, including learning rate, LSTM units, dropout rate, and batch size. This could further improve model performance beyond manual tuning.

7.4 Final Selected Hyperparameters

After experimentation, the best-performing configuration was selected as shown in Table 1.

Hyperparameter	**Selected Value**
LSTM layers	2
Units	$128 \rightarrow 64$
Dropout	0.2
Batch size	32
Optimizer	Adam
Learning rate	0.001
Loss function	Mean Squared Error (MSE)
Early stopping patience	5

Table 1: Final selected hyperparameters for the LSTM model.

7.5 Impact on Model Performance

Hyperparameter tuning improved the final LSTM model by reducing overfitting, stabilizing validation loss, and improving generalization on test data.

Performance Improvements:

- Validation loss became more stable during training.
- Dropout and early stopping reduced overfitting.
- The final model achieved improved MAE, RMSE, and R^2 performance.
- The final R^2 score reached approximately 0.7787, showing that the model explained about 77.87% of the variation in appliance energy consumption.

8 Model Evaluation

Evaluation Metrics:

8.1 Mean Absolute Error (MAE)

Measures average prediction error.

8.2 Root Mean Squared Error (RMSE)

Penalizes larger prediction errors.

8.3 Mean Absolute Percentage Error (MAPE)

Measures percentage prediction error.

8.4 R-Squared (R^2)

Measures variance explained by the model.

9 Results & Discussion

The final LSTM model demonstrated strong predictive performance. The model performance results are summarized in Table 2.

Model	MAE	RMSE	R^2
LSTM Model	0.07266294196368497	0.11134569041986538	0.7786579996785451

Table 2: Final LSTM model performance results.

Model Performance:

- MAE: 0.07266294196368497
- RMSE: 0.11134569041986538
- R^2: 0.7786579996785451

9.1 Key Findings

- Time-series features significantly improve model performance.
- Lag features are highly important for prediction accuracy.
- The deep learning LSTM model produced strong forecasting results for appliance energy prediction.

9.2 Insight

Energy consumption is highly dependent on temporal and environmental factors. The model achieved a Mean Absolute Error (MAE) of 0.0727 and a Root Mean Squared Error (RMSE) of 0.1113, indicating relatively low prediction error. The R^2 score of 0.7787 shows that the model explains approximately 77.87% of the variation in appliance energy consumption.

10 Visualization Results

This section presents the main visual outputs used to analyze model behavior and dataset patterns.

10.1 Energy Consumption Over Time

Figure 1 shows how appliance energy consumption changes across the recorded time period. This graph helps identify fluctuations, peaks, and possible temporal patterns in energy usage.

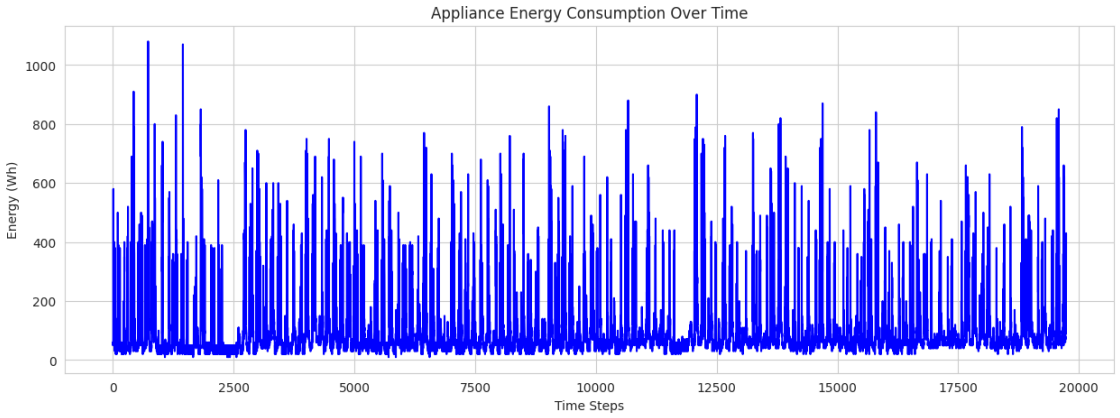

Figure 1: Energy consumption over time.

10.2 Monthly Energy Trend

Figure 2 shows the monthly pattern of appliance energy consumption. This graph helps identify month-to-month variation and broader seasonal changes in energy usage.

10.3 Temperature vs Energy Consumption

Figure 3 illustrates the relationship between temperature and appliance energy consumption. This visualization helps determine whether changes in temperature are associated with higher or lower energy demand.

10.4 Feature Correlation Heatmap

Figure 4 presents the correlation between the main numerical features in the dataset. This visualization helps identify relationships among temperature, humidity, weather, and appliance energy variables.

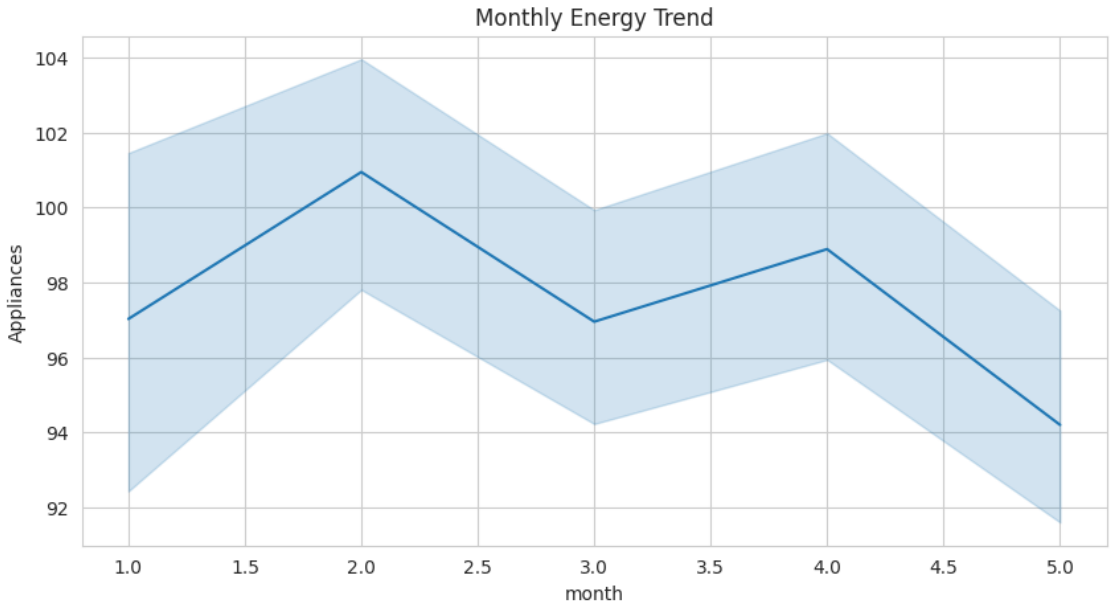

Figure 2: Monthly energy consumption trend.

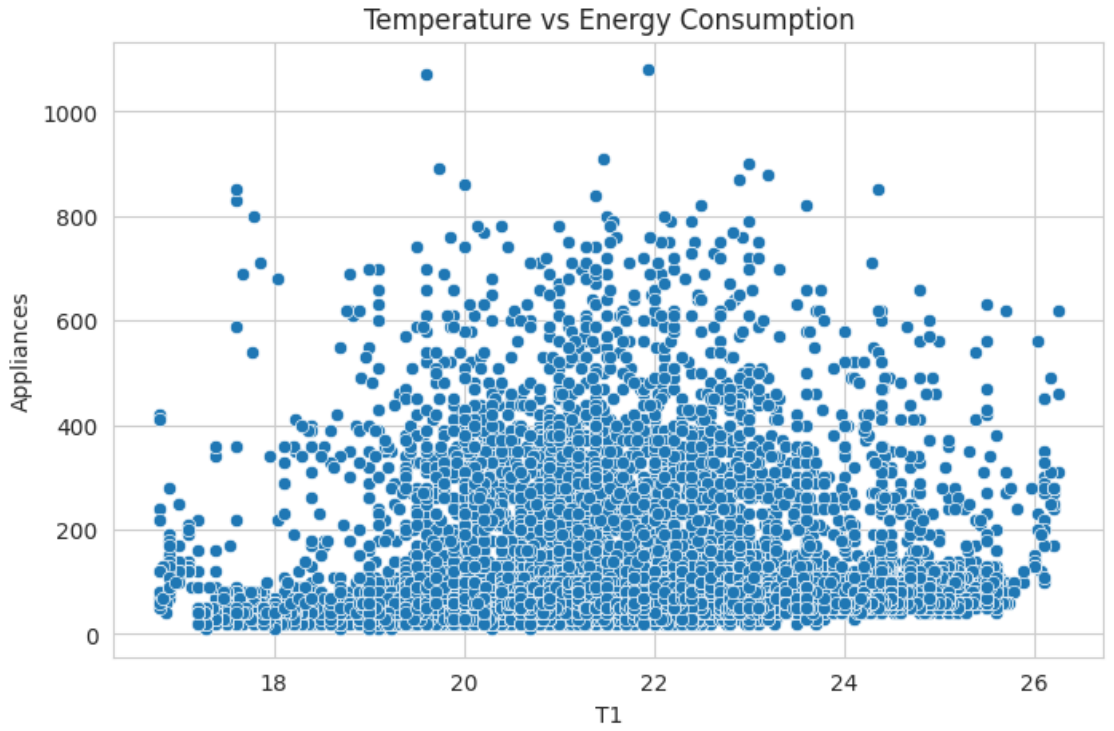

Figure 3: Temperature vs energy consumption.

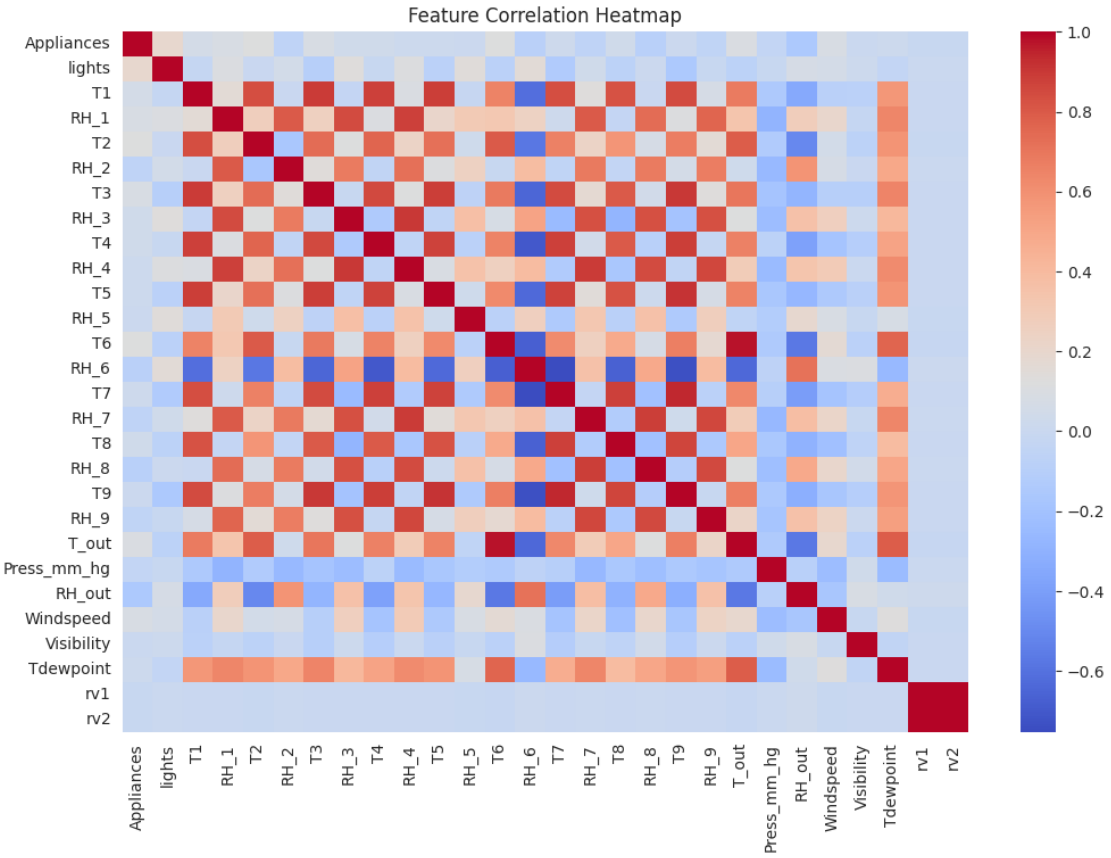

Figure 4: Feature correlation heatmap.

10.5 Training vs Validation Loss

Figure 5 compares the training and validation loss during LSTM model training. This graph is useful for checking whether the model is learning effectively or overfitting.

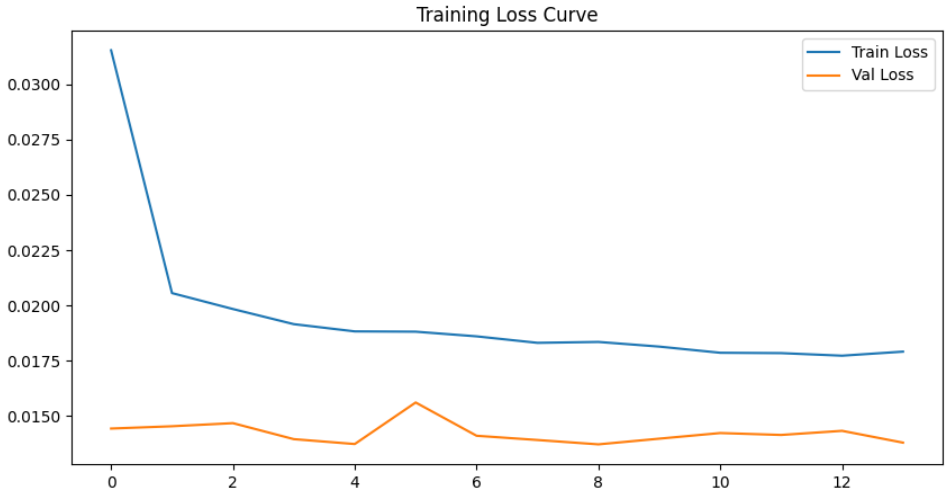

Figure 5: Training vs validation loss for the LSTM model.

10.6 Actual vs Predicted Energy Consumption

Figure 6 compares actual appliance energy consumption values with predicted values. This graph helps evaluate how closely the model predictions follow the real energy consumption trend.

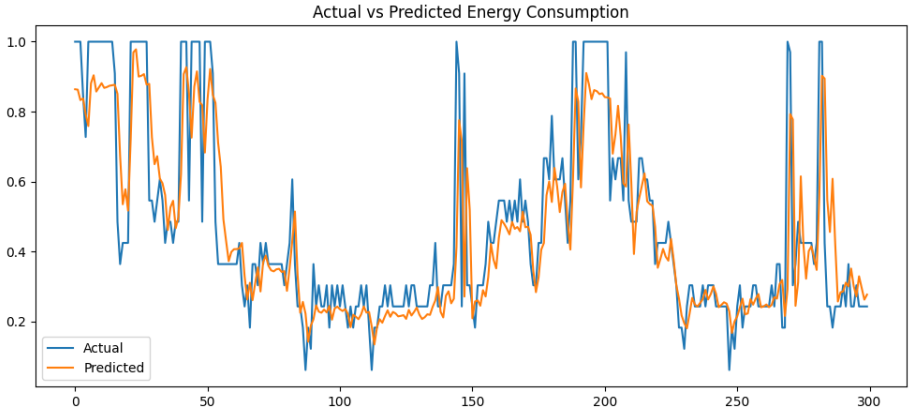

Figure 6: Actual vs predicted appliance energy consumption.

11 Challenges and Solutions

11.1 Challenge 1

Handling temporal dependencies.

Solution:

Lag features and LSTM networks were implemented.

11.2 Challenge 2

Overfitting.

Solution:

Dropout regularization and early stopping were applied.

11.3 Challenge 3

High-dimensional feature space.

Solution:

Feature selection was performed to retain the most relevant predictors.

12 Conclusion

This project successfully developed a deep learning model for appliance energy consumption prediction.

Key Findings:

- Feature engineering significantly improved predictive performance.
- The optimized LSTM achieved strong forecasting accuracy.
- Hyperparameter tuning and regularization improved model generalization.

Future Work:

- Experiment with GRU networks.
- Explore CNN-LSTM hybrid architectures.
- Incorporate external weather forecasts and holiday information.
- Apply advanced optimization techniques such as Bayesian Optimization.

13 References

Reference Materials:

- **Time Series Forecasting:**
  - Time Series Forecasting with LSTM Neural Networks (TensorFlow Tutorial)
- **Feature Engineering for Time Series:**
  - Kaggle – Time Series Feature Engineering
- **Deep Learning Guides:**
  - *Deep Learning with Python* – François Chollet
  - PyTorch Official Tutorials

A Appendix

A.1 GitHub Repository

The complete project source code, notebooks, and related implementation files are available in the GitHub repository below:

- GitHub Repository: `https://github.com/Jithara14/Appliance-Energy-Prediction.git`